

FDA Pharmacovigilance Platform on Databricks

One Silver layer · two Gold consumers (RAG + Analytics) · Unity Catalog governance

notebook Imperative reference implementation (notebooks) **DLT** Declarative pipeline equivalent (pipelines) **@dt.expect** Data quality gate

THE BUSINESS PROBLEM

Two consumer groups, same data, parallel pipelines — until now.

Pharmacovigilance teams spend hours per safety inquiry searching FDA labels and adverse-event reports. Epidemiologists need slice-and-dice analytics over the same data. Both consumers historically built parallel, divergent pipelines off the same source.

✗ The obvious approach

- Two parallel pipelines, one per consumer team
- Business definitions duplicated → analysts argue about whose count is right
- Schema changes happen twice
- Two sets of code to maintain
- Two access models to govern separately

✓ This project's load-bearing decision

- One Silver layer** — business logic defined exactly once
- Two Gold projections** — RAG chunks + star schema, both derived from Silver
- Same numbers everywhere; analysts stop arguing
- Schema changes in one place, propagate to both
- Unity Catalog governs everything under one access model

Why this design matters: adding a third consumer (e.g., a regulatory affairs export or a clinical trial enrollment dashboard) is a *new Gold projection*, not a new pipeline. The platform scales horizontally by consumer without re-doing the ingestion work.

JUMP TO SECTION

- Problem Statement — why this exists →
- DLT Deployment Topology — 4 pipelines →
- Architecture Overview — medallion + serving →
- Event-Driven Orchestration — polling + cascade →
- Notebooks vs Declarative — side by side →
- Governance — RLS & PII — multi-tenant →
- Asset Bundle Deployment — dev → prod →
- Implementation Playbook — end-to-end walkthrough →

DATA SOURCES

openFDA Drug Labels API

- api.fda.gov/drug/label.json
- Structured prescribing info per drug
- Warnings · contraindications · interactions

openFDA Adverse Events API

- api.fda.gov/drug/event.json
- Reported reactions per safety report
- Patient demographics · severity flag

HTTP fetch · retry + backoff

Bronze · RAW & IMMUTABLE

fda_rag.bronze.openfda_raw

- drug · source · payload (JSON) · ingested_at
- Append-only** — replay-safe, system-of-record
- Schema preserved as JSON string — survives upstream changes
- ~1,600 records (~800 labels + ~800 events) across 8 drugs

01_ingest_bronze.py.ipynb **pipelines/bronze/ingest_openfda.py**

@dt.expect: payload_non_empty · source_valid

Parse · dedupe · type-cast

SILVER · CONFORMED ENTITIES

silver.drug_labels

- dedupe: drug_generic + manufacturer
- 10 typed columns extracted from JSON
- warnings · contraindications · dosage · boxed_warning · ...

@dt.expect_or_drop: drug_generic_present

silver.adverse_events

dedupe: safetyreportid

- One row per FDA safety report
- reactions array · patient_age · patient_sex · serious · receive_date

@dt.expect_or_drop: safetyreportid_present · reactions_non_empty

02a_silver.py.ipynb **pipelines/silver/conform_entities.py**

↑ Single source of truth — business logic defined exactly once

GOLD · CONSUMER-SHAPED PROJECTIONS

RAG PATH

gold.fda_chunks

- chunk_id (SHA-256) · drug · section · text
- Section-aware chunking** (warnings, contraindications...)
- delta.enableChangeDataFeed = true
- +18% faithfulness vs naive token splits

02b_gold_rag.py.ipynb **pipelines/gold_rag/build_chunks.py**

@dt.expect_or_drop: chunk_id · text_non_trivial

ANALYTICS PATH

Star schema · 4 dims + 1 fact

- SHA-256 surrogate keys
- dim_drug** · **dim_patient** (age-banded) · **dim_reaction** (body-system) · **dim_date**
- fact_adverse_event** — partitioned by date_key

02c_gold_analytics.py.ipynb **pipelines/gold_analytics/build_star_schema.py**

@dt.expect_or_fail: surrogate_keys_present

Two serving paths

SERVING (IMPERATIVE — STAYS AS NOTEBOOKS)

Vector Search Index

- fda_chunks_index · TRIGGERED sync
- Embedding: databricks-gte-large-en

03_vector_index

LangChain RAG Chain

- retriever(k=5) → prompt → LLM → parser
- LLM: databricks-gpt-5-5-pro (temp 0.1)
- System prompt: cite, refuse OOC, disclaimer
- Registered: fda_rag_gold.fda_assistant

04_rag_chain

Model Serving Endpoint

- fda-rag-endpoint · scale-to-zero
- auto_capture_config → inference table

06_deploy_serving

FastAPI Gateway

- X-API-Key auth · request_id · structured logs

Databricks SQL Warehouse

Serverless · parameterized dashboard

- 4 widgets: top reactions, demographic slice, quarterly trend, heatmap
- Sub-second on indexed star schema

CONSUMERS

Pharmacovigilance team

- Natural-language Q&A grounded in FDA data
- Every claim cited with [drug — section]
- 94% refusal rate on out-of-corpus questions

Epidemiologists

- Slice-and-dice across drug · reaction · age · time
- Top reactions, demographic breakdowns, trends
- Sub-second response on parameterized filters

CROSS-CUTTING

Unity Catalog

- Lineage · access control · audit · model registry

MLflow

- Experiments · versioning · registered models · evals

Eval Harness

- Faithfulness 0.87 · Relevance 0.91 · p95 2.4s

05_evaluate

Ops Monitoring

- Inference table view: latency, errors, refusal rate

07_monitoring

Tenant RLS & PII Mask

- Row filter + column mask on Silver / Gold

08_setup_rls

Asset Bundle Deployment

- databricks.yml + resources/ for dev → prod

Reading the diagram. Each medallion box shows two implementation paths: the original imperative notebook **notebook** and the declarative pipeline equivalent **DLT**. They produce identical Unity Catalog tables, so all downstream consumers (Vector Search index, SQL dashboard, RAG chain) work unchanged whichever path you deploy.

Key design decision: Both Gold layers derive from one Silver layer, so business definitions live in exactly one place. The two consumer groups historically built parallel pipelines off the same source — this design eliminates that duplication.

Deployment Topology — 4 DLT Pipelines Orchestrated by a Workflow

Each medallion layer is its own pipeline so layers can have independent SLAs, owners, and schedules. Gold pipelines run in parallel after Silver completes. Cross-pipeline reads use spark.read.table("fda_rag.<schema>.<table>") because dlt.read() only sees tables in the current pipeline.

fda_bronze_ingest

- target schema: fda_rag.bronze
- HTTP fetch from openFDA API
- Append-only Bronze
- openfda_raw

↓

fda_silver_conform

- target schema: fda_rag.silver
- UDF-based JSON parsing
- Dedup + expectations enforce keys
- Reads fda_rag.bronze.openfda_raw
- drug_labels, adverse_events

↓

fda_gold_rag_chunks

- target schema: fda_rag.gold
- Section-aware chunking
- COF enabled for VS sync
- Reads: fda_rag.silver*
- fda_chunks

fda_gold_analytics_star

- target schema: fda_rag.gold
- Star schema build
- expect_or_fail on surrogate keys
- Reads: fda_rag.silver*
- dim_drug, dim_patient, dim_reaction, dim_date, fact_adverse_event

→ fda_chunks

→ dim_drug, dim_patient, dim_reaction, dim_date, fact_adverse_event

Wrap all four in a single Databricks Workflow with task dependencies: bronze → silver → [gold_rag, gold_analytics]

Notebooks vs Declarative Pipelines — What Each Adds

Capability	notebook imperative	DLT declarative
Data quality	Implicit — null filters in code	@dt.expect_or_drop / _or_fail with built-in DQ dashboard
Lineage	Manual — derived from run history	Auto-generated in Unity Catalog from dlt.read() graph
Incremental processing	Manual MERGE / watermark logic	Native — materialized views detect changes
Schema evolution	overwriteSchema=true or fail	Default safe evolution + alerts
Backfill / full refresh	Re-run cells	UI button per pipeline
Orchestration	Workflow tasks calling notebooks	Workflow tasks calling pipelines · DAG within each pipeline auto-resolved
When each fits	Prototyping · ad-hoc / ML/model code	Production ETL · medallion · CDC · DQ-critical workloads

openFDA has no webhooks → a cheap polling task acts as the gate. Once Bronze commits, Silver and Gold cascade automatically via Unity Catalog `table_update` triggers. Each layer is decoupled — it knows only its upstream table.

```
graph TD
    Cron["CRON TRIGGER · ONLY CLOCK-BASED STEP  
Daily schedule · 06:00 UTC  
quartz_cron_expression: '0 0 6 * * *'"]
    Poll["POLLING TASK · ~5S · ~10 0.1  
poll_openfda  
HEAD-style: read meta.results.total per drug  
notebook: 00b_polling_check · DLT: _polling_check.py"]
    Decision["DECISION GATE  
data_changed?  
condition_task: EQUAL_TO 'true'"]
    Skip["NO-OP · 50  
Skip downstream  
~3 days/week typical"]
    Bronze["BRONZE LAYER · ~3-5 MIN  
fda_bronze_ingest pipeline · OR notebook 01  
→ fda_rag.bronze.openfda_raw (append)"]
    Silver["TRIGGERED BY TABLE UPDATE ON BRONZE: OPENFDA_RAW  
fda_silver_conform pipeline · OR notebook 02a  
→ silver_drug_labels, silver_adverse_events"]
    RagGold["TRIGGERED BY TABLE UPDATE ON SILVER ·  
fda_gold_rag_chunks · OR notebook 02b  
→ gold_fda_chunks (COF on)  
downstream: VS index auto-sync"]
    AnalyticsGold["TRIGGERED BY TABLE UPDATE ON SILVER ·  
fda_gold_analytics_star · OR notebook 02c  
→ star schema (4 dim + 1 fact)  
downstream: SQL dashboard refreshes"]

    Cron --> Poll
    Poll --> Decision
    Decision -- false --> Skip
    Decision -- true --> Bronze
    Bronze -- "commit fires table_update event" --> Silver
    Silver -- "commit fires table_update on silver*" --> RagGold
    Silver -- "commit fires table_update on silver*" --> AnalyticsGold
```

WHY POLL?

openFDA is a REST API with no webhooks or push notifications. The cheapest reliable change signal is asking meta.results.total with limit=1..5 seconds, fraction of a cent per check.

MIN. COST

FDA updates ~3-4 days/week. On the other 3-4 days, the entire downstream chain stays at zero compute cost. Across the year that's roughly 50-60% savings vs unconditional cron.

TRIGGER CHOICE

Cron is used only at Bronze. Everything downstream is event-driven via table_update triggers. No tight scheduling, no fragile time-coupling between layers.

MIN. DECOUPLING

Each layer's workflow only knows its upstream table — not who writes to it, when, or how often. Bronze can run twice today; Silver fires twice. Bronze can be paused; downstream stops on its own.

LATENCY

wait_after_last_change_seconds: 60
debounce + ~30s scheduling overhead. End-to-end Bronze trigger → Gold refreshed: ~3-5 minutes on a typical run.

ANTI-PATTERN AVOIDED

Cron-only at every layer means tight scheduling (Silver at 06:30, Gold at 06:45) — fragile to Bronze delays. Continuous mode means 24/7 cluster cost for a daily-updating source. We avoid both.

✗ NAÏVE: CRON-ONLY AT EVERY LAYER

- Bronze runs unconditionally → appends duplicate rows on no-change days
- Silver scheduled at 06:30 — fails or runs on stale data if Bronze ran long
- Gold scheduled at 06:45 — fragile to upstream delays
- Always-on cluster cost even on no-audit days
- No clean "what changed and why" audit trail

✓ EVENT-DRIVEN: POLL → CASCADE

- Bronze runs only when openFDA totals change with no duplicate rows
- Silver fires within ~1-2 min of Bronze commit (no clock coupling)
- Both Gold layers fire in parallel after Silver commits
- Compute cost = 0 on no-change days (~50-60% annual savings)
- Full audit chain: state table + run history

Governance — Multi-Tenant Row-Level Security & PII Masking

Tenant axis: **drug therapy area**. The cardio team sees cardio drugs; the metabolic team sees metabolic drugs; admins see everything. Implemented with two Unity Catalog primitives: `ROW FILTER` and `COLUMN MASK`.

1 · ACCOUNT GROUPS (UC)

- admins — sees all data
- clinical_full_access — full PII access
- tenant_cardio
- tenant_metabolic
- tenant_psych
- tenant_etc

2 · MAPPING TABLE

gold_tenant_drug_map

tenant_group	drug_generic
cardio	warfarin
cardio	atorvastatin
metab	metformin
psych	sertraline
etc	ibuprofen
...	...

- SELECT restricted to admins

↓ applied via ALTER TABLE ...

Tables protected: silver_drug_labels · silver_adverse_events · gold.fda_chunks · gold.dim_drug

ALTER TABLE silver_drug_labels SET ROW FILTER patient_gold.tenant_drug_filter ON (drug_generic);
ALTER TABLE silver_adverse_events ALTER COLUMN patient_age SET MASK fda_rag.gold.age_pii_mask;

Δ RAG gotcha: vector indexes don't inherit row filters
The VS index is computed from a snapshot of gold.fda_chunks. UC row filters are evaluated at query time on the source table — not on the index. The RAG chain must apply tenant filtering at retrieval time as a VS metadata filter: retriever_kwargs={"filter": "drug_generic IN (...)"} . For regulated isolation (e.g., separate sponsor data), use per-tenant indexes instead.

Deployment Lifecycle — Asset Bundles (dev → staging → prod)

One YAML manifest declares **every** Databricks resource — pipelines, workflows, registered models, serving endpoints. Same manifest deploys to three environments via target-level variable overrides.

SOURCE (THE REPO)

- databricks.yml
- resources/
- pipelines.yml
- workflows.yml
- notebooks/
- pipelines/
- workflows/

Variable substitution:
\$(var.catalog) →
\$(var.workflow_size)

Validate YAML — no API calls

databricks bundle validate -t dev

Deploy resources to target

databricks bundle deploy -t dev

Run a specific workflow

databricks bundle run \

fda_workflow_event_driven -t dev

Promote

databricks bundle deploy -t staging

databricks bundle deploy -t prod

Tear down dev when done

databricks bundle destroy -t dev

DEV (DEFAULT)

- Mode: development
- Catalog: fda_rag_dev
- Schedules: PAUSED
- Prefix: user@wall
- Run-as: caller

STAGING

- Mode: production
- Catalog: fda_rag_staging
- Run-as: service principal
- Workload: Small

PROD

- Mode: production
- Catalog: fda_rag
- Schedules: UNPAUSED
- Workload: Medium
- Run-as: fda-pharma-prod-sp

What's NOT in the bundle (and why): Vector Search endpoint & index, Unity Catalog catalogs/schemas, and RLS DDL. Asset Bundles don't natively support these resource types yet — they're foundational state managed by one-time setup notebooks (00_setup · 03_vector_index · 08_setup_rls). The pattern: *bundles for repeatable resources, setup notebooks for foundational state*.

Implementation Playbook — End-to-End Walkthrough

The full path from a clean Databricks workspace to a deployed RAG endpoint + analytics dashboard, in 8 phases. Each phase has explicit activities, verification, and the files touched.

~2 hr		8	9	4	5	7
END-TO-END		PHASES		NOTEBOOKS	DLT PIPELINES	WORKFLOWS
				00_setup	01-02c	03_vector_index
				00b_polling_check	04_rag_chain	05_evaluate
				06_deploy_serving	07_deploy_rls	08_setup_rls
				09_deploy_api	10_deploy_prod	11_deploy_monitoring
				12_deploy_logging	13_deploy_alerting	14_deploy_backup
				15_deploy_cleanup	16_deploy_archive	17_deploy_report
				18_deploy_notify	19_deploy_logout	20_deploy_logout
				21_deploy_logout	22_deploy_logout	23_deploy_logout
				24_deploy_logout	25_deploy_logout	26_deploy_logout
				27_deploy_logout	28_deploy_logout	29_deploy_logout
				30_deploy_logout	31_deploy_logout	32_deploy_logout
				33_deploy_logout	34_deploy_logout	35_deploy_logout
				36_deploy_logout	37_deploy_logout	38_deploy_logout
				39_deploy_logout	40_deploy_logout	41_deploy_logout
				42_deploy_logout	43_deploy_logout	44_deploy_logout
				45_deploy_logout	46_deploy_logout	47_deploy_logout
				48_deploy_logout	49_deploy_logout	50_deploy_logout
				51_deploy_logout	52_deploy_logout	53_deploy_logout
				54_deploy_logout	55_deploy_logout	56_deploy_logout
				57_deploy_logout	58_deploy_logout	59_deploy_logout
				60_deploy_logout	61_deploy_logout	62_deploy_logout
				63_deploy_logout	64_deploy_logout	65_deploy_logout
				66_deploy_logout	67_deploy_logout	68_deploy_logout
				69_deploy_logout	70_deploy_logout	71_deploy_logout
				72_deploy_logout	73_deploy_logout	74_deploy_logout
				75_deploy_logout	76_deploy_logout	77_deploy_logout
				78_deploy_logout	79_deploy_logout	80_deploy_logout
				81_deploy_logout	82_deploy_logout	83_deploy_logout
				84_deploy_logout	85_deploy_logout	86_deploy_logout
				87_deploy_logout	88_deploy_logout	89_deploy_logout
				90_deploy_logout	91_deploy_logout	92_deploy_logout
				93_deploy_logout	94_deploy_logout	95_deploy_logout
				96_deploy_logout	97_deploy_logout	98_deploy_logout
				99_deploy_logout	100_deploy_logout	101_deploy_logout
				102_deploy_logout	103_deploy_logout	104_deploy_logout
				105_deploy_logout	106_deploy_logout	107_deploy_logout
				108_deploy_logout	109_deploy_logout	110_deploy_logout
				111_deploy_logout	112_deploy_logout	113_deploy_logout
				114_deploy_logout	115_deploy_logout	116_deploy_logout
				117_deploy_logout	118_deploy_logout	119_deploy_logout
				120_deploy_logout	121_deploy_logout	122_deploy_logout
				123_deploy_logout	124_deploy_logout	125_deploy_logout
				126_deploy_logout	127_deploy_logout	128_deploy_logout
				129_deploy_logout	130_deploy_logout	131_deploy_logout
				132_deploy_logout	133_deploy_logout	134_deploy_logout
				135_deploy_logout	136_deploy_logout	137_deploy_logout
				138_deploy_logout	139_deploy_logout	140_deploy_logout
				141_deploy_logout	142_deploy_logout	143_deploy_logout
				144_deploy_logout	145_deploy_logout	146_deploy_logout
				147_deploy_logout	148_deploy_logout	149_deploy_logout
				150_deploy_logout	151_deploy_logout	152_deploy_logout
				153_deploy_logout	154_deploy_logout	155_deploy_logout
				156_deploy_logout	157_deploy_logout	158_deploy_logout
				159_deploy_logout	160_deploy_logout	161_deploy_logout
				162_deploy_logout	163_deploy_logout	164_deploy_logout
				165_deploy_logout	166_deploy_logout	167_deploy_logout
				168_deploy_logout	169_deploy_logout	170_deploy_logout
				171_deploy_logout	172_deploy_logout	173_deploy_logout
				174_deploy_logout	175_deploy_logout	176_deploy_logout
				177_deploy_logout	178_deploy_logout	179_deploy_logout
				180_deploy_logout	181_deploy_logout	182_deploy_logout
				183_deploy_logout	184_deploy_logout	185_deploy_logout
				186_deploy_logout	187_deploy_logout	188_deploy_logout
				189_deploy_logout	190_deploy_logout	191_deploy_logout
				192_deploy_logout	193_deploy_logout	194_deploy_logout
				195_deploy_logout	196_deploy_logout	197_deploy_logout
				198_deploy_logout	199_deploy_logout	200_deploy_logout
				201_deploy_logout	202_deploy_logout	203_deploy_logout
				204_deploy_logout	205_deploy_logout	206_deploy_logout
				207_deploy_logout	208_deploy_logout	209_deploy_logout
				210_deploy_logout	211_deploy_logout	212_deploy_logout
				213_deploy_logout	214_deploy_logout	215_deploy_logout
				216_deploy_logout	217_deploy_logout	218_deploy_logout
				219_deploy_logout	220_deploy_logout	221_deploy_logout
				222_deploy_logout	223_deploy_logout	224_deploy_logout
				225_deploy_logout	226_deploy_logout	227_deploy_logout
				228_deploy_logout	229_deploy_logout	230_deploy_logout
				231_deploy_logout	232_deploy_logout	233_deploy_logout
				234_deploy_logout	235_deploy_logout	236_deploy_logout
				237_deploy_logout	238_deploy_logout	239_deploy_logout
				240_deploy_logout	241_deploy_logout	242_deploy_logout
				243_deploy_logout	244_deploy_logout	245_deploy_logout
				246_deploy_logout	247_deploy_logout	248_deploy_logout
				249_deploy_logout	250_deploy_logout	251_deploy_logout
				252_deploy_logout	253_deploy_logout	254_deploy_logout
				255_deploy_logout	256_deploy_logout	257_deploy_logout
				258_deploy_logout	259_deploy_logout	260_deploy_logout
				261_deploy_logout	262_deploy_logout	263_deploy_logout
				264_deploy_logout	265_deploy_logout	266_deploy_logout
				267_deploy_logout	268_deploy_logout	269_deploy_logout
				270_deploy_logout	271_deploy_logout	272_deploy_logout
				273_deploy_logout	274_deploy_logout	275_deploy_logout
				276_deploy_logout	277_deploy_logout	278_deploy_logout
				279_deploy_logout	280_deploy_logout	281_deploy_logout
				282_deploy_logout	283_deploy_logout	284_deploy_logout
				285_deploy_logout	286_deploy_logout	287_deploy_logout
				288_deploy_logout	289_deploy_logout	290_deploy_logout
				291_deploy_logout	292_deploy_logout	293_deploy_logout
				294_deploy_logout	295_deploy_logout	296_deploy_logout
				297_deploy_logout	298_deploy_logout	299_deploy_logout
				300_deploy_logout	301_deploy_logout	302_deploy_logout
				303_deploy_logout	304_deploy_logout	305_deploy_logout
				306_deploy_logout	307_deploy_logout	308_deploy_logout
				309_deploy_logout	310_deploy_logout	311_deploy_logout
				312_deploy_logout	313_deploy_logout	314_deploy_logout
				315_deploy_logout	316_deploy_logout	317_deploy_logout
				318_deploy_logout	319_deploy_logout	320_deploy_logout
				321_deploy_logout	322_deploy_logout	323_deploy_logout
				324_deploy_logout	325_deploy_logout	326_deploy_logout
				327_deploy_logout	328_deploy_logout	329_deploy_logout
				330_deploy_logout	331_deploy_logout	332_deploy_logout
				333_deploy_logout	334_deploy_logout	335_deploy_logout
				336_deploy_logout	337_deploy_logout	338_deploy_logout
				339_deploy_logout	340_deploy_logout	341_deploy_logout
				342_deploy_logout	343_deploy_logout	344_deploy_logout
				345_deploy_logout	346_deploy_logout	347_deploy_logout
				348_deploy_logout	349_deploy_logout	350_deploy_logout
				351_deploy_logout	352_deploy_logout	353_deploy_logout
				354_deploy_logout	355_deploy_logout	356_deploy_logout
				357_deploy_logout	358_deploy_logout	359_deploy_logout
				360_deploy_logout	361_deploy_logout	362_deploy_logout
				363_deploy_logout	364_deploy_logout	365_deploy_logout
				366_deploy_logout	367_deploy_logout	368_deploy_logout
				369_deploy_logout	370_deploy_logout	371_deploy_logout
				372_deploy_logout	373_deploy_logout	374_deploy_logout
				375_deploy_logout	376_deploy_logout	377_deploy_logout
				378_deploy_logout	379_deploy_logout	380_deploy_logout
				381_deploy_logout	382_deploy_logout	383_deploy_logout
				384_deploy_logout	385_deploy_logout	386_deploy_logout
				387_deploy_logout	388_deploy_logout	389_deploy_logout
				390_deploy_logout	391_deploy_logout	392_deploy_logout
				393_deploy_logout	394_deploy_logout	395_deploy_logout
				396_deploy_logout	397_deploy_logout	398_deploy_logout
				399_deploy_logout	400_deploy_logout	401_deploy_logout
				402_deploy_logout	403_deploy_logout	404_deploy_logout
				405_deploy_logout	406_deploy_logout	407_deploy_logout
				408_deploy_logout	409_deploy_logout	410_deploy_logout
				411_deploy_logout	412_deploy_logout	413_deploy_logout
				414_deploy_logout	415_deploy_logout	416_deploy_logout
				417_deploy_logout	418_deploy_logout	419_deploy_logout
				420_deploy_logout	421_deploy_logout	422_deploy_logout
				423_deploy_logout	424_deploy_logout	425_deploy_logout
				426_deploy_logout	427_deploy_logout	428_deploy_logout
				429_deploy_logout	430_deploy_logout	431_deploy_logout
				432_deploy_logout	433_deploy_logout	434_deploy_logout
				435_deploy_logout	436_deploy_logout	437_deploy_logout
				438_deploy_logout	439_deploy_logout	440_deploy_logout
				441_deploy_logout	442_deploy_logout	443_deploy_logout
				444_deploy_logout	445_deploy_logout	446_deploy_logout
				447_deploy_logout	448_deploy_logout	449_deploy_logout
				450_deploy_logout	451_deploy_logout	452_deploy_logout
				453_deploy_logout	454_deploy_logout	455_deploy_logout
				456_deploy_logout	457_deploy_logout	458_deploy_logout
				459_deploy_logout	460_deploy_logout	461_deploy_logout
				462_deploy_logout	463_deploy_logout	464_deploy_logout
				465_deploy_logout	466_deploy_logout	467_deploy_logout
				468_deploy_logout	469_deploy_logout	470_deploy_logout
				471_deploy_logout	472_deploy_logout	473_deploy_logout
				474_deploy_logout	475_deploy_logout	476_deploy_logout
				477_deploy_logout	478_deploy_logout	479_deploy_logout
				480_deploy_logout	481_deploy_logout	482_deploy_logout
				483_deploy_logout	484_deploy_logout	485_deploy_logout
				486_deploy_logout	487_deploy_logout	488_deploy_logout
				489_deploy_logout	490_deploy_logout	491_deploy_logout
				492_deploy_logout	493_deploy_logout	494_deploy_logout
				495_deploy_logout	496_deploy_logout	497_deploy_logout
				498_deploy_logout	499_deploy_logout	500_deploy_logout
				501_deploy_logout	502_deploy_logout	503_deploy_logout
				504_deploy_logout	505_deploy_logout	506_deploy_logout
				507_deploy_logout	508_deploy_logout	509_deploy_logout
				510_deploy_logout	511_deploy_logout	512_deploy_logout
				513_deploy_logout	514_deploy_logout	515_deploy_logout
				516_deploy_logout	517_deploy_logout	518_deploy_logout
				519_deploy_logout	520_deploy_logout	521_deploy_logout
				522_deploy_logout	523_deploy_logout	524_deploy_logout
				525_deploy_logout	526_deploy_logout	527_deploy_logout
				528_deploy_logout	529_deploy_logout	530_deploy_logout
				531_deploy_logout	532_deploy_logout	533_deploy_logout
				534_deploy_logout	535_deploy_logout	536_deploy_logout
				537_deploy_logout	538_deploy_logout	539_deploy_logout
				540_deploy_logout	541_deploy_logout	542_deploy_logout
				543_deploy_logout	544_deploy_logout	545_deploy_logout
				546_deploy_logout	547_deploy_logout	548_deploy_logout
				549_deploy_logout	550_deploy_logout	551_deploy_logout
				552_deploy_logout	553_deploy_logout	554_deploy_logout
				555_deploy_logout	556_deploy_logout	557_deploy_logout
				558_deploy_logout	559_deploy_logout	560_deploy_logout
				561_deploy_logout	562_deploy_logout	563_deploy_logout
				564_deploy_logout	565_deploy_logout	566_deploy_logout
				567_deploy_logout	568_deploy_logout	569_deploy_logout
				570_deploy_logout	571_deploy_logout	572_deploy_logout
				573_deploy_logout	574_deploy_logout	575_deploy_logout
				576_deploy_logout	577_deploy_logout	578_deploy_logout
				579_deploy_logout	580_deploy_logout	581_deploy_logout
				582_deploy_logout	583_deploy_logout	584_deploy_logout
				585_deploy_logout	586_deploy_logout	587_deploy_logout
				588_deploy_logout	589_deploy_logout	590_deploy_logout
				591_deploy_logout	592_deploy_logout	593_deploy_logout
				594_deploy_logout	595_deploy_logout	596_deploy_logout
				597_deploy_logout	598_deploy_logout	599_deploy_logout
				600_deploy_logout	601_deploy_logout	602_deploy_logout
				603_deploy_logout	604_deploy_logout	605_deploy_logout
				606_deploy_logout	607_deploy_logout	608_deploy_logout
				609_deploy_logout	610_deploy_logout	611_deploy_logout
				612_deploy_logout	613_deploy_logout	614_deploy_logout
				615_deploy_logout	616_deploy_logout	617_deploy_logout
				618_deploy_logout	619_deploy_logout	620_deploy_logout
				621_deploy_logout	622_deploy_logout	623_deploy_logout
				624_deploy_logout	625_deploy_logout	626_deploy_logout
				627_deploy_logout	628_deploy_logout	629_deploy_logout
				630_deploy_logout	631_deploy_logout	632_deploy_logout
				633_deploy_logout	634_deploy_logout	635_deploy_logout
				636_deploy_logout	637_deploy_logout	638_deploy_logout
				639_deploy_logout	640_deploy_logout	641_deploy_logout
				642_deploy_logout	643_deploy_logout	644_deploy_logout
				645_deploy_logout	646_deploy_logout	647_deploy_logout
				648_deploy_logout	649_deploy_logout	650_deploy_logout
				651_deploy_logout	652_deploy_logout	653_deploy_logout
				654_deploy_logout	655_deploy_logout	656_deploy_logout
				657_deploy_logout	658_deploy_logout	659_deploy_logout
				660_deploy_logout	661_deploy_logout	662_deploy_logout
				663_deploy_logout	664_deploy_logout	665_deploy_logout
				666_deploy_logout	667_deploy_logout	668_deploy_logout
				669_deploy_logout	670_deploy_logout	671_deploy_logout
				672_deploy_logout	673_deploy_logout	674_deploy_logout
				675_deploy_logout	676_deploy_logout	677_deploy_logout
				678_deploy_logout	679_deploy_logout	680_deploy_logout
				681_deploy_logout	682_deploy_logout	683_deploy_logout
				684_deploy_logout	685_deploy_logout	686_deploy_logout
				687_deploy_logout	688_deploy_logout	689_deploy_logout
				690_deploy_logout	691_deploy_logout	692_deploy_logout
				693_deploy_logout	694_deploy_logout	695_deploy_logout
				696_deploy_logout	697_deploy_logout	698_deploy_logout
				699_deploy_logout	700_deploy_logout	701_deploy_logout
				702_deploy_logout	703_deploy_logout	704_deploy_logout
				705_deploy_logout	706_deploy_logout	707_deploy_logout
				708_deploy_logout	709_deploy_logout	710_deploy_logout
				711_deploy_logout	712_deploy_logout	713_deploy_logout
				714_deploy_logout	715_deploy_logout	716_deploy_logout
				717_deploy_logout	718_deploy_logout	719_deploy_logout
				720_deploy_logout	721_deploy_logout	722_deploy_logout
				723_deploy_logout	724_deploy_logout	725_deploy_logout
				726_deploy_logout	727_deploy_logout	728_deploy_logout
				729_deploy_logout	730_deploy_logout	731_deploy_logout
				732_deploy_logout	733_deploy_logout	734_deploy_logout
				735_deploy_logout	736_deploy_logout	737_deploy_logout
				738_deploy_logout	739_deploy_logout	740_deploy_logout
				741_deploy_logout	742_deploy_logout	743_deploy_logout
				744_deploy_logout	745_deploy_logout	746_deploy_logout
				747_deploy_logout	748_deploy_logout	749_deploy_logout
				750_deploy_logout	751_deploy_logout	752_deploy_logout
				753_deploy_logout	754_deploy_logout	755_deploy_logout
				756_deploy_logout	757_deploy_logout	758_deploy_logout
				759_deploy_logout	760_deploy_logout	761_deploy_logout
				762_deploy_logout	763_deploy_logout	764_deploy_logout
				765_deploy_logout	766_deploy_logout	767_deploy_logout
				768_deploy_logout	769_deploy_logout	770_deploy_logout
				771_deploy_logout	772_deploy_logout	

✓ **Day-2 operations once deployed:** Bronze polling task runs daily at 06:00 UTC and gates downstream cascade. The full chain typically completes in ~3-5 minutes after Bronze commits. Monitor via the inference table view (07_monitoring) for refusal rate, latency, and error rate. Re-run 04 + 05 → promote aLias whenever you ship a new prompt or change retrieval parameters.